

Claude Code Memory Retrieval Layer

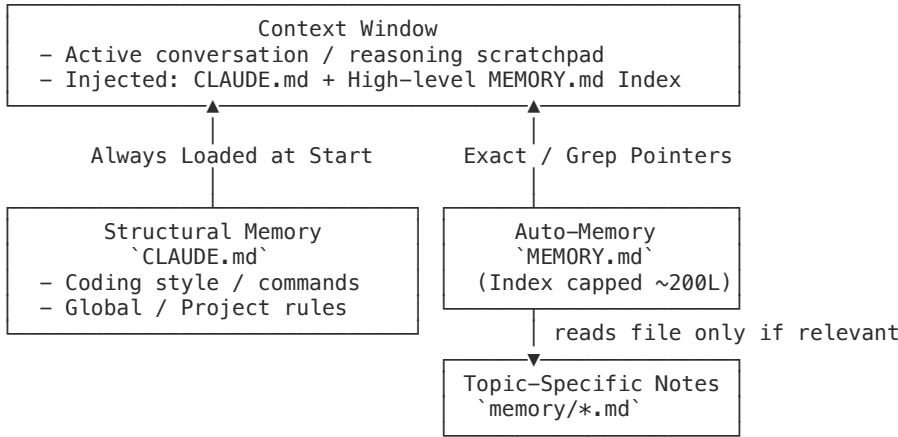
https://gemini.google.com/app/a1054a37ddd02ff3

User prompt: what is a general retrieval layer over your memory in Claude Code

Response: A **retrieval layer over memory in Claude Code** refers to the architecture that indexes, searches, and selectively injects saved project knowledge, guidelines, and historical session notes into the agent's context window instead of dumping everything into context all at once.

1. How Claude Code's Native Memory Retrieval Works

Rather than relying on heavy vector databases, Claude Code's built-in memory system uses a file-based, tiered architecture designed to be token-efficient, auditable, and locally inspectable:



- Static Ingestion (CLAUDE.md / AGENTS.md):** Loaded directly at session startup (global ~/ .claude/CLAUDE.md and repository-level). It provides persistent rules, commands, and architecture guidelines.
- Index-and-Pointer Layer (MEMORY.md):** Auto-generated notes live under ~/ .claude/projects/<project>/memory/. MEMORY.md serves as an index (capped around 200 lines) containing brief one-line pointers (under ~150 characters each) to dedicated Markdown detail files.
- Lookup & Retrieval Mechanism:**
 - Grep / Keyword Matching:** When Claude needs specific historical context, it scans the index using keyword matching/regex, retrieves the matching sub-file, and loads only that relevant file into the conversation.
 - Verification Requirement:** The internal prompt instructs the model to treat retrieved memories as **hints, not authoritative truths**. It verifies assumptions against the codebase before modifying code.
- Maintenance / Consolidation ("Auto Dream"):** Background sub-agent passes periodically consolidate redundant notes, resolve contradictions, convert relative dates to absolute timestamps, and prune stale lines to keep the index compact.

2. General External Retrieval Layers (Semantic / Hybrid)

Because Claude Code's built-in retrieval relies primarily on strict keyword lookups and local file boundaries, developers and third-party tools (such as **memsearch** or custom MCP servers) implement **generalized retrieval layers** over Claude Code's memory:

Component	Native Claude Code	General External Retrieval Layer
Search Engine	Exact substring / Grep	Hybrid (Semantic dense vectors + BM25 sparse matching)
Vocabulary Mismatch	Fails if search wording differs from note	Solves synonyms (e.g., "Redis timeout" maps to "socket drop")
Storage Backend	Plain local Markdown directory	Vector DB (e.g., Milvus, Qdrant, Chroma, or SQLite-vss)
Tool Integration	Native read/write tool calls	Custom MCP (Model Context Protocol) tools or background skills
Portability	Locked to Claude Code project scope	Shared across multiple agents (Claude Code, OpenCode, Codex, etc.)

3. Why This Separation Matters

- Protects Context Budget:** Prevents loading megabytes of past session logs into the token window.
- Minimizes "Instruction Drift":** Keeping long-term memories externalized and retrieved *just-in-time* stops old, outdated edge-case notes from distorting everyday completions.

- **Maintains Ground Truth:** Decouples persistent facts from ephemeral agent scratchpads so you can inspect, edit, or wipe memories in simple plaintext without restarting your workflow.

Sources

1. [Claude Code Source Leak: The Three-Layer Memory Architecture](#)
 2. [Claude Code Memory System Explained: 4 Layers, 5 Limits, and a Fix](#)
-

User prompt: How do I set up a custom MCP server to provide semantic vector search over Claude Code's memory files?

Response: To provide semantic vector search over Claude Code's memory files, you can build a lightweight local MCP server using Python (fastmcp and chromadb or sentence-transformers) that monitors your memory files and registers it with Claude Code via stdio.

Claude Code stores auto-memory files locally under `~/ .claude/projects/<project-slug>/memory/`.

Step 1: Set Up the Environment

Create a dedicated directory for your server and install the necessary dependencies:

```
mkdir -p ~/claude-memory-mcp && cd ~/claude-memory-mcp
python3 -m venv venv
source venv/bin/activate
```

```
# Install FastMCP, an embedded vector store, and a local embedding engine
pip install "fastmcp>=0.1.0" chromadb sentence-transformers
```

Step 2: Create the MCP Server Script

Create `server.py` in `~/claude-memory-mcp/server.py`. This script monitors the Claude Code memory directories, indexes Markdown chunks, and exposes a `search_memory` tool:

```
import glob
import os
from pathlib import Path
import chromadb
from chromadb.utils import embedding_functions
from fastmcp import FastMCP

# Initialize FastMCP server
mcp = FastMCP("claude-memory-search")

# Persistent ChromaDB storage inside the MCP server folder
DB_DIR = os.path.expanduser("~/claude-memory-mcp/data")
os.makedirs(DB_DIR, exist_ok=True)

# Local sentence-transformer embeddings (no external API key needed)
embed_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)

client = chromadb.PersistentClient(path=DB_DIR)
collection = client.get_or_create_collection(
    name="claude_code_memories",
    embedding_function=embed_fn,
    metadata={"hnsw:space": "cosine"}
)

def chunk_markdown(content: str, max_chars: int = 600) -> list[str]:
    """Splits markdown content into paragraph-based chunks."""
    paragraphs = content.split("\n\n")
    chunks = []
    current_chunk = []
    current_len = 0

    for p in paragraphs:
        p_clean = p.strip()
        if not p_clean:
            continue
        if current_len + len(p_clean) > max_chars and current_chunk:
            chunks.append("\n\n".join(current_chunk))
            current_chunk = [p_clean]
            current_len = len(p_clean)
        else:
            current_chunk.append(p_clean)
            current_len += len(p_clean)
```

```

        current_chunk.append(p_clean)
        current_len += len(p_clean)

    if current_chunk:
        chunks.append("\n\n".join(current_chunk))
    return chunks

def sync_memory_files():
    """Scans and synchronizes all memory Markdown files across Claude Code projects."""
    memory_root = Path(os.path.expanduser("~/claude/projects"))
    files = glob.glob(str(memory_root / "**" / "memory" / "*.md"), recursive=True)

    for file_path in files:
        file_path_obj = Path(file_path)
        mtime = os.path.getmtime(file_path)
        rel_project = file_path_obj.parent.parent.name
        doc_id_prefix = f"{rel_project}:{file_path_obj.name}"

        try:
            with open(file_path, "r", encoding="utf-8") as f:
                content = f.read()
        except Exception:
            continue

        chunks = chunk_markdown(content)
        for idx, chunk in enumerate(chunks):
            chunk_id = f"{doc_id_prefix}:chunk_{idx}"
            collection.upsert(
                documents=[chunk],
                metadatas=[{
                    "source_path": file_path,
                    "project": rel_project,
                    "file_name": file_path_obj.name,
                    "mtime": mtime
                }],
                ids=[chunk_id]
            )

@mcp.tool()
def search_memory(query: str, project_filter: str = "", top_k: int = 5) -> str:
    """
    Search past learnings, architecture decisions, and session notes stored in memory.

    Args:
        query: Semantic query (e.g., 'database connection pool settings', 'docker build bug').
        project_filter: Optional substring to restrict results to a specific project.
        top_k: Number of relevant snippets to return.
    """
    # Sync on lookup to catch any new or edited notes
    sync_memory_files()

    where_filter = None
    if project_filter:
        where_filter = {"project": {"$contains": project_filter}}

    results = collection.query(
        query_texts=[query],
        n_results=min(top_k, max(collection.count(), 1)),
        where=where_filter
    )

    if not results or not results["documents"] or not results["documents"][0]:
        return "No relevant memories found."

    output = []
    for doc, meta in zip(results["documents"][0], results["metadatas"][0]):
        output.append(
            f"""**File:** `{meta['file_name']}` (Project: `{meta['project']}`)\n"
            f"""**Path:** `{meta['source_path']}`\n"
            f"""**Excerpt:**\n{doc}\n"
        )

    return "\n---\n".join(output)

```

```
if __name__ == "__main__":
    # Runs over standard input/output transport
    mcp.run(transport="stdio")
```

Step 3: Register the Server with Claude Code

Register your server as a `stdio` transport tool using the CLI. You can register it at the **user scope** so it is accessible across all your repositories:

```
claude mcp add --transport stdio --scope user memory-search \
  -- ~/claude-memory-mcp/venv/bin/python ~/claude-memory-mcp/server.py
```

Verify that Claude Code recognizes the server and reports a valid connection:

```
claude mcp list
```

You should see:

```
✓ memory-search: Connected
```

Step 4: Direct Claude Code to Use the Retrieval Layer

To ensure Claude queries this tool before making assumptions or trying brute-force file searches, add a rule to your global `~/.claude/CLAUDE.md` or repo-level `CLAUDE.md`:

```
# Memory & Retrieval
- You have access to a semantic search tool named `memory-search`.
- Before searching through old conversation logs or assuming codebase conventions, call `search_memory` with :
```

Testing the Setup

1. Launch a Claude Code session:

```
claude
```

2. Prompt Claude to verify the integration:

"Run a search using `memory-search` to see if we have any past notes about our Redis cache setup."

3. Claude will execute `memory-search:search_memory`, embed your query, rank the Markdown snippets in `~/.claude/projects/*/memory/`, and cite the exact file and note without filling the context window.

Sources

1. [Add MCP Servers to Claude Code - Setup & Configuration Guide](#)
 2. [Claude Code has an undocumented persistent memory feature](#)
-